



Санкт-Петербургский государственный университет
Кафедра системного программирования

Разработка драйвера для RISC-V N-Trace подсистемы

Патов Артемий Сергеевич, группа 25.M41-мм

Научный руководитель: к.ф.-м.н. В. И. Гориховский, доцент кафедры системного программирования
Консультант: А. М. Разинков, руководитель группы, ООО «КНС Групп»

Санкт-Петербург
2026

Введение: RISC-V N-Trace

- Аппаратная трассировка обеспечивает глубокий анализ исполнения (instruction/data flow) без влияния на производительность.
- **RISC-V N-Trace** основан на стандарте Nexus (IEEE 5001). Обеспечивает гибкую систему кодирования событий и фильтрации.
- Мощные возможности маршрутизации данных в аппаратные стоки (SRAM, System Memory) или их экспорт (например, через PCIe).

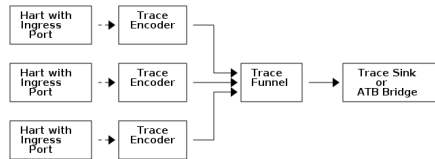


Рис.: Компоненты потока trace-данных

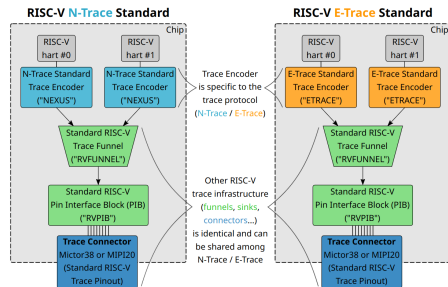


Рис.: Существующие протоколы кодировки

Intel Processor Trace (PT)

Широко распространенный стандарт для x86. Высокая пропускная способность, но относительно слабоконфигурируемая система фильтрации на аппаратном уровне. Полностью полагается на стандартный интерфейс perf API.

ARM CoreSight ETM

Продвинутая инфраструктура макроячеек (Embedded Trace Macrocell). Архитектурно значительно ближе к RISC-V N-Trace, предлагая гибкие механизмы маршрутизации через ATB (AMBA Trace Bus).

Инфраструктура ARM CoreSight

Ключевые концепции CoreSight

- CoreSight ETM использует стандартизированную шину AMBA Trace Bus (ATB) для маршрутизации потоков.
- Трассировочные данные объединяются через воронки (Trace Funnels) и направляются в аппаратные стоки (ETB/TPIU).
- Архитектурная парадигма RISC-V N-Trace крайне близка к ARM Coresight, что позволяет переиспользовать многие концепции.

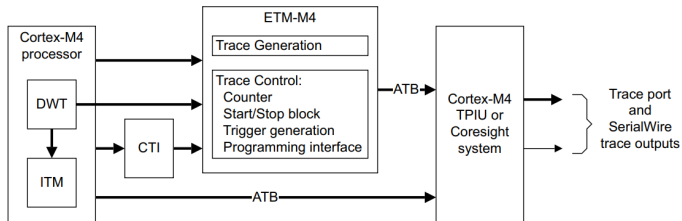


Рис.: Блок-схема Coresight ETM

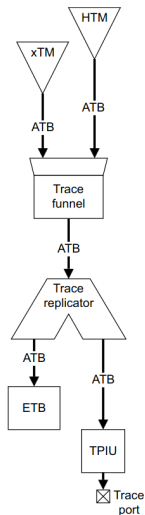


Рис.: Инфраструктура CoreSight

Подсистема N-Trace на SoC

- Интеграция с существующей инфраструктурой ATB позволяет переиспользовать маршрутизаторы (Funnel) и стоки (Sink).
- Существенно упрощает внедрение на кристаллах, уже имеющих элементы экосистемы CoreSight.

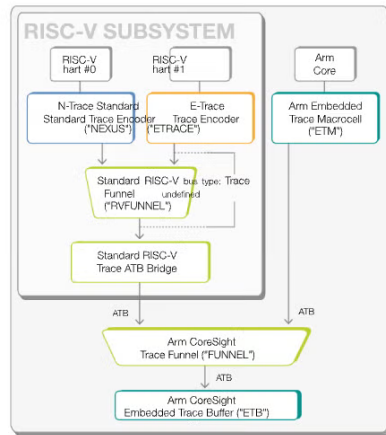


Figure 4: Trace infrastructure for mixed RISC-V/Arm SoCs.
©Lauterbach

Постановка задачи

Целью дипломной работы является разработка Linux-драйвера для системы трассировки N-Trace в архитектуре RISC-V, обеспечивающего интеграцию с подсистемой профилирования perf и предоставляющего механизмы конфигурации аппаратных компонентов трассировки.

Задачи на текущий семестр:

- Исследование спецификаций RISC-V Trace Control, N-Trace и анализ аналогов (Intel PT, ARM CoreSight ETM).
- Портирование драйвера rvtrace для целевой платформы и устранение выявленных багов.
- Поддержка фильтрации по потоку инструкций и идентификаторам процессов:
 - ▶ Адаптация поддержки регистров контекста (scontext / hcontext) для идентификации процессов ОС.
 - ▶ Поддержка независимой настройки компараторов и аппаратных фильтров.
 - ▶ Интеграция фильтров с интерфейсом подсистемы perf.

Драйвер rvtrace v4

Интеграция открытого драйвера (патч `riscv_trace_support_v4`), реализующего минимальную функциональность (Enable/Disable + интеграция с perf) в проприетарный форк Linux v7.0.
Устранены некоторые исходные баги.

Регистры контекста

Портирован патч для `scontext/hcontext`.
Исправлены серьезные баги (например, запись в верхние 32 бита `sstateen`).
Регистр `scontext` теперь успешно применяется для идентификации процессов ОС.

Архитектура фильтрации N-Trace

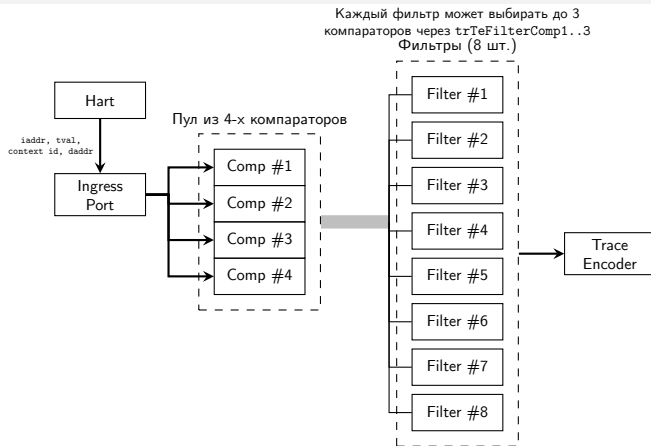


Рис.: Архитектура фильтрации N-Trace: сигналы от Hart поступают в Ingress Port, распределяются по всем компараторам, результаты которых передаются по общей шине. Каждый из 8 фильтров может настраиваемым образом подключаться к любому из 4 компараторов (или нескольким), используя регистры выбора.

Интеграция с perf-tool и PMU-ивентами

- N-Trace регистрируется в подсистеме ядра как PMU. Настройки из userspace мапятся на структуру `perf_event_attr`.
- Строка фильтра из `perf-tool` парсится ядром и конвертируется в настройки пула компараторов N-Trace и фильтров.

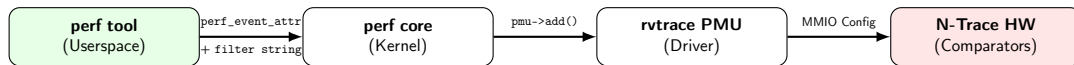


Рис.: Взаимодействие компонентов при передаче конфигурации от perf к аппаратному обеспечению

Пример запуска трассировки с аппаратным фильтром:

```
perf record -e rvtrace/event=0x1/uk \  
--filter 'filter_0xffff_0xffffffff'
```

Intel PT

Целиком полагается на конфигурацию, предоставляемую через perf API. Используются базовые адресные фильтры, режимы привилегий и 64 бита метаданных. Система проста, но ограничена.

ARM CoreSight ETM

Предлагает обширную конфигурацию в обход perf, используя подгружаемые модули ядра или configs. Архитектурно ближе к N-Trace, но использование модулей ядра для загрузки конфигураций создает риски безопасности.

Стратегия поддержки фильтрации

Фаза 1: Базовая (perf API)

Интеграция фильтров через стандартный интерфейс perf API. Полная совместимость со стандартными утилитами. Конфигурация надежна, но ограничена базовыми возможностями perf_event_attr. (Реализовано)

Фаза 2: Полноценная (Bypass)

Глубокая настройка в обход perf API с использованием sysfs. Позволяет безопасно раскрыть полный потенциал аппаратных компараторов и фильтров N-Trace.

Реализация Фазы 1

- Разделение сущностей: фильтры и компараторы полностью настраиваются внутри модуля `rvtrace-encoder`.
- Драйвер энкодера предоставляет API для передачи конфигурации извне.
- В модуле `rvtrace-perf` происходит парсинг аргументов `perf` API и трансформация во внутренние структуры энкодера.

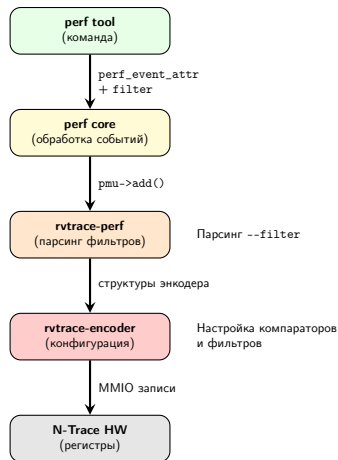


Рис.: Поток конфигурации в Фазе 1

- Детально изучены спецификации **Intel PT** и **ARM CoreSight ETM**. Выбрана ориентация на архитектуру CoreSight.
- Успешно перенесен в проприетарный форк и доработан открытый патч минимального драйвера rvtrace.
- Портирован и исправлен патч поддержки scontext, обеспечена идентификация процессов ОС.
- Поддержана независимая настройка фильтров и компараторов в драйвере rvtrace-encoder, Обеспечена интеграция адресных фильтров с интерфейсом подсистемы perf.